Lab session 5: Programming Fundamentals

Problem 1: Equalizer

Consider an array of n non-negative integers. Two adjacent elements are called *balanced* if their values differ by at most one.
The array is modified by repeatedly applying the following operation:

- Find the leftmost element that is at least two larger than one of its adjacent elements.
- Decrease this element by one and increase its smallest adjacent element by one. If both adjacent elements have the same value, choose the left one.

The first and last elements of the array have only one adjacent element, which is therefore their only possible neighbor for the transfer. The process stops when all adjacent elements are balanced.

For example, the array `[2,5,1]` changes as follows:

$$[2, 5, 1] \rightarrow [2, 4, 2] \rightarrow [3, 3, 2].$$

The first line of the input contains an integer n ($2 \leq n \leq 10^4$). The second line contains n non-negative integers a_i (where $0 \leq a_i \leq 1000$), representing the initial contents of the array.

The output must contain the final contents of the array in the form $[a_1, a_2, \ldots, a_n]$, with the elements separated by commas and no spaces.

Example 1:
input:
```
3
2 5 1
```
output:
```
[3,3,2]
```

Example 2:
input:
```
5
10 3 10 9 3
```
output:
```
[7,8,7,7,6]
```

Example 3:
input:
```
10
9 8 10 4 8 6 8 9 6 0
```
output:
```
[9,8,7,7,7,7,7,6,5,5]
```

Problem 2: Enigma Machine

The Enigma machine was a famous encryption device used during World War II. In this exercise, we implement a simplified version of the Enigma machine. The machine consists of three components:

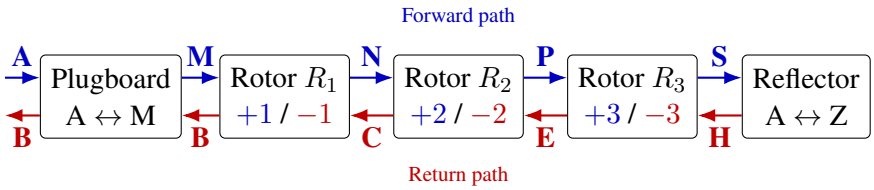
- **Plugboard:** Pairs of letters can be connected, causing them to be exchanged. For example, if A and B are connected, A is replaced by B and B by A. Letters that are not connected remain unchanged.
- **Rotors:** The machine contains r rotors, numbered $R_1, \ldots, R_r$. Each rotor has a setting between 0 and 25. On the forward path, a rotor shifts a letter forward by its setting, wrapping around from Z to A. On the return path, the same rotor shifts the letter backward by its setting.

- **Reflector:** The reflector replaces each letter by its opposite in the alphabet: A $\leftrightarrow$ Z, B $\leftrightarrow$ Y, . . ., M $\leftrightarrow$ N.

Each letter first passes through the plugboard, then through the rotors in order $R_1, \ldots, R_r$, and finally reaches the reflector. It then returns through the rotors in reverse order, $R_r, \ldots, R_1$, and passes through the plugboard once more.

All shifts wrap around the alphabet. For example, shifting Z forward by 2 produces B, while shifting A backward by 2 produces Y.

The following figure illustrates how the letter A is encoded using three rotors with settings $[1, 2, 3]$ and a plugboard connecting A and M.



The letter A is first exchanged with M by the plugboard. The rotors then shift M to N, P, and S. The reflector replaces S by H, after which the rotors shift H back to E, C, and B. Since B is not connected on the plugboard, the final output is B.

After encoding each letter, the rotor settings change:

- Rotor R_1 increases by 1, wrapping from 25 back to 0.
- Whenever a rotor wraps from 25 to 0, the next rotor also increases by 1.
- This carry mechanism continues through the rotors as necessary.

For example, with three rotors: $[24, 7, 3] \to [25, 7, 3] \to [0, 8, 3]$. Similarly, $[25, 25, 3] \to [0, 0, 4]$.

The initial rotor settings are used to encode the first letter. The rotors are updated only after the letter has passed through the entire machine. In the example above, after encoding A as B, the rotor settings change from $[1, 2, 3]$ to $[2, 2, 3]$.

The input consists of four lines:

1. A nonempty string of uppercase letters (A–Z) to encode.
2. Two integers k and r, where $0 \leq k \leq 13$ and $1 \leq r < 10^4$. These specify the number of plugboard pairs and the number of rotors, respectively.
3. The k plugboard pairs, each represented by two distinct uppercase letters separated from other pairs by spaces. No letter occurs in more than one pair. If $k = 0$, this line is empty.
4. The r initial rotor settings, each an integer between 0 and 25, in order $R_1, \ldots, R_r$.

The output has the same format as the input, but the string on the first input line is replaced by its encoded version. The plugboard pairs and initial rotor settings must remain unchanged.

Note: The encryption is reversible. Encoding an encrypted string again, using the same plugboard connections and initial rotor settings, produces the original string.

Example 1:

input:

```
ALANTURING
2 3
AM RS
1 2 3
```

output:

```
BMXUAJJSKP
2 3
AM RS
1 2 3
```

Example 2:

input:

```
BMXUAJJSKP
2 3
AM RS
1 2 3
```

output:

```
ALANTURING
2 3
AM RS
1 2 3
```

Example 3:

input:

```
VHZHBYSCWZFINN
3 5
DC RS AM
2 3 5 7 11
```

output:

```
MARIANREJEWSKI
3 5
DC RS AM
2 3 5 7 11
```

Problem 3: Lagrange's Four Squares

Lagrange's four-square theorem states that every non-negative integer n can be expressed as the sum of four squares:

$$n = a^2 + b^2 + c^2 + d^2,$$

where a, b, c, and d are non-negative integers.

Given an integer n (where $0 \leq n \leq 10^6$) on the input, your program should find four integers a, b, c, and d satisfying

$$a \geq b \geq c \geq d \geq 0 \quad \text{and} \quad n = a^2 + b^2 + c^2 + d^2.$$

If multiple solutions exist, output the **lexicographically smallest** one: the solution with the smallest value of a. If several solutions have the same a, choose the one with the smallest b, then c, and finally d. You are required to print the lexicographically smallest solution in the format `a^2+b^2+c^2+d^2`, without spaces.

Example 1:

input:
```
4242
```
output:
```
34^2+34^2+33^2+29^2
```

Example 2:

input:
```
1
```
output:
```
1^2+0^2+0^2+0^2
```

Example 3:

input:
```
27831
```
output:
```
89^2+86^2+83^2+75^2
```

Problem 4: Bernoulli's Triangle

Bernoulli's triangle is a variation of Pascal's triangle. Its rows are numbered starting from 1. The first element of each row is 1, and the last element of row n is 2^{n-1}. Every element in between is the sum of the two elements directly above it.
The first five rows are:

$$\begin{array}{ccccccccc}
 & & & & 1 & & & & \\
 & & & 1 & & 2 & & & \\
 & & 1 & & 3 & & 4 & & \\
 & 1 & & 4 & & 7 & & 8 & \\
1 & & 5 & & 11 & & 15 & & 16
\end{array}$$

More formally, let $B_{n,k}$ denote the k-th element of row n, where $1 \leq k \leq n$. Then

$$B_{n,k} = \begin{cases} 1 & \text{if } k = 1, \\ B_{n-1,k-1} + B_{n-1,k} & \text{if } 1 < k < n, \\ 2^{n-1} & \text{if } k = n. \end{cases}$$

Since the values grow rapidly, all elements must be computed and printed **modulo** m. The input consists of three integers a, b, and m, satisfying $1 \leq a \leq b \leq 10^4$ and $1 < m \leq 10^9$.

The output must consist of the pyramid rows a through b (inclusive), one row per line. The field width is determined by the number of decimal digits in $m - 1$, regardless of the actual values printed. Each number is right-aligned in a field of the specified width, using # characters for

padding. For example, if $m = 11$, the field width is 2, and the number 4 is printed as `#4`, while 42 is printed as `42`. Numbers on a row are separated by a single space character.

Example 1
Input:
```
1 5 2
```
Output:
```
1
1 0
1 1 0
1 0 1 0
1 1 1 1 0
```

Example 2
Input:
```
4 7 11
```
Output:
```
#1 #4 #7 #8
#1 #5 #0 #4 #5
#1 #6 #5 #4 #9 10
#1 #7 #0 #9 #2 #8 #9
```

Example 3
Input:
```
2 2 10000
```
Output:
```
###1 ###2
```

Problem 5: Powerful

Given an integer n, compute 2^n and print the result in scientific notation with **perfect accuracy**: no rounding or truncation is allowed. Scientific notation represents a number as $a \times 10^k$, where $1 \leq a < 10$ and k is an integer.

Print the result in the format `a*10^k`, where a is written in decimal notation without unnecessary trailing zeros or a trailing decimal point.

The input consists of a single integer n, where $-10000 \leq n \leq 10000$. The output must be the exact value of 2^n in the specified scientific notation. All digits must be printed, without rounding or truncation. Note that, due to the large range of n, standard floating-point arithmetic cannot be used to obtain the required accuracy.

Example 1
Input:
```
10
```
Output:
```
1.024*10^3
```

Example 2
Input:
```
-1
```
Output:
```
5*10^-1
```

Example 3
Input:
```
-3
```
Output:
```
1.25*10^-1
```